

iOS Weather.app

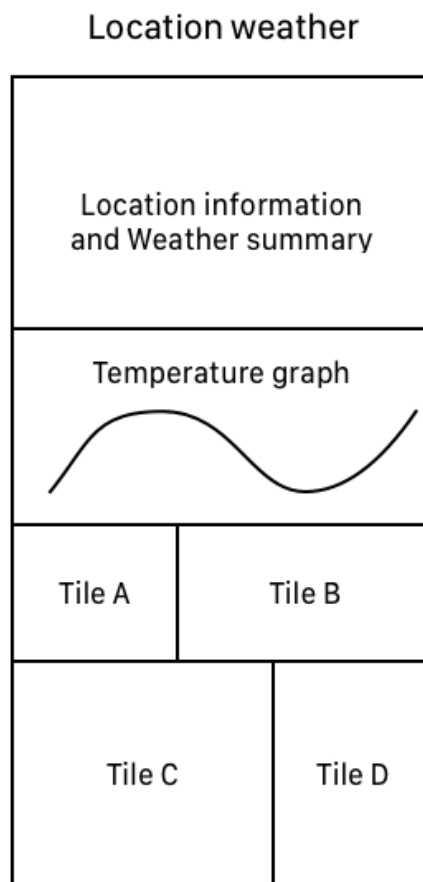
Dear developer,

We would like to ask you develop a Weather.app.

Please go through all pages in this document. Especially, don't miss to read the technical requirements mentioned below.

A. Here is a description of App:

1. App consists primary with one screen - "Location weather" screen.



2. "Location weather" screen has an arbitrary view at a top followed by collection view at a middle and arbitrary view at the bottom of the screen.
3. Top view of "Location weather" screen includes information about current location and weather summary.
4. Middle view backed by collection view represents hourly forecast with a temperature graph.
5. Bottom view represents "Tiles", similar to "Microsoft Windows 10" start menu tiles. Each tile represents arbitrary information about location and weather summary.

B. Here is a how “Location weather” screen works:

Use iOS SDK API to lookup coordinates of the current location. Then use latitude and longitude coordinates to fetch weather information from weather data provider (see “Technical details” section below).

Allow user to refresh weather data by performing pull gesture. This feature widely known as a “pull-to-refresh”.

1. Location and Weather summary

Show on top of the screen brief information about location, such as location name, country name or country code, zip code, level above sea, etc.

Also show brief information about weather at that location, such as current temperature, minimum and maximum temperatures for the current day, air pollution, pressure, humidity, etc.

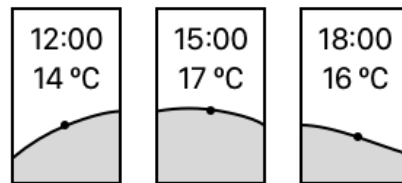
When there is enough space on screen (e.g. iPad in full screen size), show more information. When there is not enough space (e.g. iPhone or small App window on iPad), then show less information to provide consistent user interface layout.

2. Temperature graph

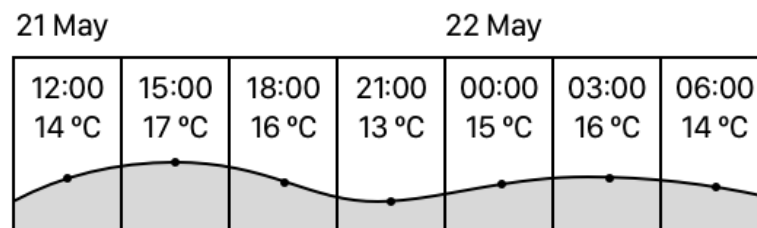
Show approximately at the middle of the screen horizontally scrollable collection view with hourly forecasts fetched from weather data provider. Each cell should represent one sample from data response. Put, at top of cell content view, information such as time, temperature, weather etc. At bottom of the cell put chunk of temperature graph, in a way, so that whole collection view row should represent an infinite graph.

The dot shown on temperature graph (see image below) represents current temperature.

Independent collection view cells



Joined collection view cells as an infinite graph



Above the cells put a collection view header with a current date. When user scrolls collection view, the header also should scroll maintaining its position relative to the first cell of the current or the next day. If during scrolling header reached left edge of the collection view, then it should stick on left side. If during scrolling header of the next day reaches position of header of the current date, then it pushes existing sticky header left occupying its position. Think about collection view header behaviour as typical sticky header behaviour in table views but positioned vertically.

3. Tiles

Fill remaining part of the screen, at a bottom, with rectangular tiles. Make these tiles look similar to “Microsoft Windows 10” start menu tiles. Display in tiles various information available from weather data provider response. It can be information about location, weather, etc.

When there is enough space on screen (e.g. iPad in full screen size), show more tiles. When there is not enough space (e.g. iPhone or small App window on iPad), then show less tiles to provide consistent user interface layout.

C. Here are the technical details:

1. Setting up weather data provider account

Go to <https://openweathermap.org> website. Register and get a free access API token. Use REST endpoints included in free plan to source weather data information. You can find REST APIs available for free usage at “Detailed price” web page <https://openweathermap.org/full-price>. This page is reachable from “Pricing > Detailed Pricing & Limits” menu.

2. Project setup

Please create Git repository and, while implementing app, regularly commit changes. We should be able to see whole git history, thus avoid squashes and large commits.

Organize source code in a way, so that every enumeration, struct, class, etc. has own file.

Beatify source code regularly and especially before submission to review.

While implementing, write source code comments explaining app functionality. We should be able to understand the source code quickly.

3. Tech stack

Do not use any third-party dependencies. Use iOS SDK APIs only.

Use Swift programming language. Do not use ObjC, C or C++.

Do not use Xib or Storyboard files. White user interface from code. Do not use SwiftUI framework for creating user interface. Use UIKit framework instead.

Do not use Combine framework and “Reactive programming” in general. Use “Imperative programming” techniques to achieve easy to understand linear execution control flow.

While implementing, apply MVVM architecture pattern.

Use Swift concurrency on "Model" (e.g. Networking) layer.

Implement “Model” and “View Model” layers as a separate Swift package or framework.

Avoid usage of Swift implicitly unwrapped optionals and force try.

Cover this Swift package or framework, which includes code of “Model” and “View Model” layers, with unit tests. Achieve coverage not less than 65 %.

4. Implementation details

In addition to iPhone, app should also support iPad. Make sure that App supports "iPad Multitasking", e.g. user should be able to resize application window on iPad. Make sure that after iPad window resize content layout remains consistent. E.g. there should not be warning messages about unsatisfied auto-layout in Xcode console, misaligned user interface elements, etc.

While implementing “Temperatures graph” use UICollectionView with custom UICollectionViewLayout. Do not use system-provided collection view layouts such as UICollectionViewFlowLayout, UICollectionViewCompositionalLayout or similar. Implement own custom layout by subclassing UICollectionViewLayout. Hint: you can cover custom collection view geometry calculation with unit tests.

While implementing “Temperatures graph”, for day sticky header, use standard supplementary view, type of UICollectionViewReusableView.

Do not reload collection view on iPad window resize, invalidate layout instead. Reload collection view only if weather data is changed, for example after data fetch.

Do not use auto-layout or UIStackView when implement “Tiles” at a bottom of the “Location weather” screen. Perform frame calculation manually. Hint: You can cover geometry calculations with unit tests.

While implementing temperature graph curve make sure that it goes through dot on a collection view cell (see image above). Dot itself must be centred horizontally. If you have difficulties building curved temperature graph, implement linear graph first. In this case connect temperature dots on a collection view cell with lines instead of curves.

Properly handle cases when user denied “Location services” usage on iOS system preferences level. If user in iOS system preferences denied “Location services” usage, then handle it properly by giving user visual feedback about missed permissions.

Make sure that App reliable scales user interface elements on “Dynamic Type” text size change.

Make sure that App has no memory leaks.

D. Bonus – Bézier curve for temperature graph

We mentioned above that, if you have difficulties building curved temperature graph, then connect temperature dots with lines. But, if you managed to finish other tasks quickly, and decided to draw temperature graph as a Bézier curve instead of lines, then it is a big plus which makes you ahead of other candidates.

E. Bonus – Reorder tiles

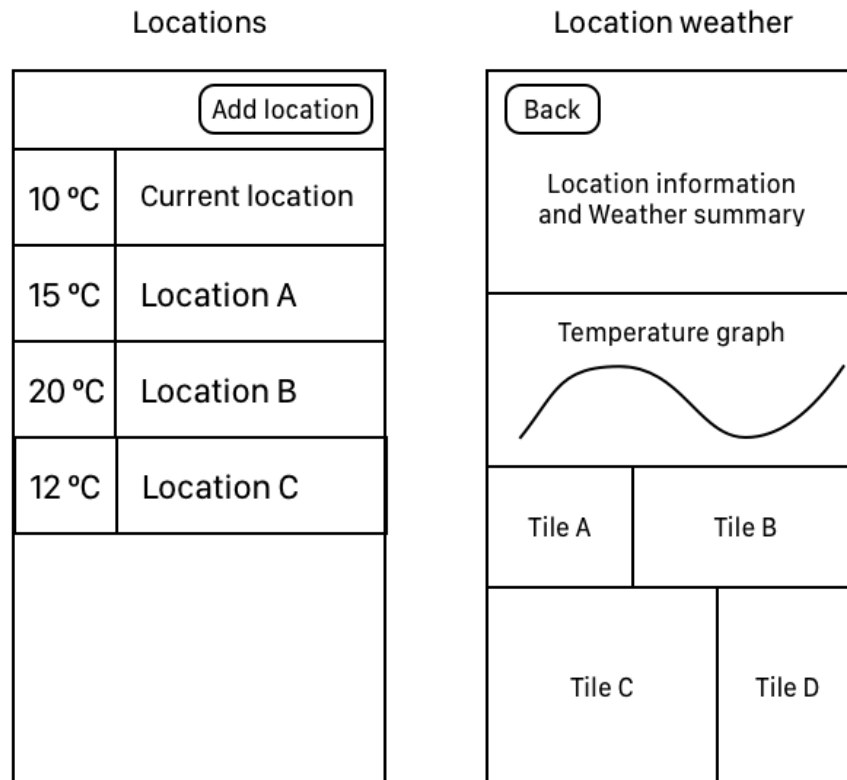
Implementing this section is optional but will be a big plus which makes you ahead of other candidates.

On “Location weather” screen, allow user to reorder tiles via drag and drop.

F. Bonus – “Locations” screen

Implementing this section is optional but will be a big plus which makes you ahead of other candidates.

In addition to “Location weather” screen implement “Locations” screen. “Locations” screen backed by navigation controller with a “Add location” navigation bar button and a table view.



1. The first row on “Locations” screen represents current location of the user. User can not remove or reorder current location. It should be always present at a first row. Remaining rows of the table view representing locations manually added by user. User can add, remove and reorder them.
2. Allow user to add new location by location name. When user taps “Add location” button, app shows an arbitrary view or view controller which allows user to specify desired location name. Use iOS SDK API to lookup location by its name and to resolve latitude and longitude coordinates. Resolved latitude and longitude coordinates later will be used to fetch weather information from weather data provider.
3. Implement “Swipe to delete” feature to allow user to remove unwanted locations.
4. Allow user to reorder previously added locations.
5. Preserve locations added by user between app restarts. Also, on app launch, automatically open recently viewed location. For doing this, implement persistence of your choice, such as UserDefaults, JSON-file on disk, PLIST-file on disk, etc.

G. Bonus – Wide screen adoption

Implementing this section is optional but will be a big plus which makes you ahead of other candidates.

This bonus requires implementation of “Locations” screen (see above).

On wide screens (e.g. iPad in landscape mode) utilize UISplitViewController to collapse or expand Master and Details view controllers. In this case “Locations” screen acts as master and “Location weather” screen acts as a details view controller.

H. Bonus – Swipe between “Location weather” screens

Implementing this section is optional but will be a big plus which makes you ahead of other candidates.

This bonus requires implementation of “Locations” screen (see above).

On “Location weather” screen, implement swipe gesture to switch between locations without need to go back to the “Locations” screen.

I. Final words

The more robust App will be, the more interactive and nicer App will be, the more stable App will be, the more chances you will have.

Good luck!